

Assignment – Module 5: Introduction to API Integration (with Python)

Name	SAGAR KUMAR
PRN	SOE25BTAM29
Section	A
Institute	Intellipaat School of Technology, Pune
Subject	Generative AI
Submission	Handwritten copy

Q1. Understand the API Request-Response Cycle

API (Application Programming Interface): a way for two programs to talk to each other. It works like a **waiter in a restaurant**: you (the client) give an order, the waiter takes it to the kitchen (the server), and brings back your food (the response).

1. **Client** – The program that **sends the request**. Example: my Python program using requests.
2. **Server** – The computer that **receives the request, processes it, and sends back data**. Example: the JSONPlaceholder server.
3. **Endpoint** – The **specific URL** where a resource is available.
Example: `https://jsonplaceholder.typicode.com/users/1`
4. **HTTP Method** – Tells the server **what action** to do:
 - **GET** – read/fetch data
 - **POST** – send/create new data
 - **PUT/PATCH** – update data
 - **DELETE** – remove data
5. **Headers** – **Extra information** about the request, sent as key-value pairs. Example: `Content-Type: application/json, Authorization: Bearer <API_KEY>`.
6. **Payload / Body** – The **actual data sent** to the server (used in POST/PUT), usually in JSON. Example: `{"name": "Sagar", "role": "Student"}`.
7. **Response** – The **reply from the server**. It contains a status code, headers and data (usually JSON).
8. **Status Code** – A **3-digit number** showing the result:
 - **200** OK (success), **201** Created
 - **400** Bad Request, **401** Unauthorized, **404** Not Found
 - **500** Internal Server Error

What happens in one cycle:

1. The client builds a request (endpoint + method + headers + body).
2. The request travels over the internet (HTTP/HTTPS) to the server.
3. The server checks it (e.g., the API key) and processes it.
4. The server sends back a **response** with a **status code** and **data**.
5. The client checks the status code, converts JSON into Python data, and uses or displays it.

Client --(Request: GET /users/1)--> Server

Client <--(Response: 200 OK + JSON)-- Server

Q2. Create and Understand JSON

```
{
  "student_name": "Sagar Kumar",
  "enrollment_year": 2025,
  "is_active": true,
  "courses": ["Generative AI", "Python Programming", "Mathematics"],
  "midterm_marks": {
    "Generative AI": 42,
    "Python Programming": 45,
    "Mathematics": 38
  },
  "final_marks": {
    "Generative AI": 85,
    "Python Programming": 90,
    "Mathematics": 78
  }
}
```

Identification:

Type	Values
String	"Sagar Kumar", and the course names "Generative AI", "Python Programming", "Mathematics"
Number	2025, 42, 45, 38, 85, 90, 78
Boolean	true (is_active)
Array	"courses": [...] – a list of 3 courses in square brackets
Nested Object	"midterm_marks": { ... } and "final_marks": { ... } – objects inside the main object

Q3. Fetch Data Using a GET Request

```
import requests

def show_user(user_id):
    url = f"https://jsonplaceholder.typicode.com/users/{user_id}"
    response = requests.get(url)          # 1. Send GET request
    print("Status Code:", response.status_code)  # 2. Print status

    if response.status_code == 200:
        data = response.json()            # 3. Convert to JSON
        (dict)
        name = data["name"]                # 4. Extract fields
        email = data["email"]
        city = data["address"]["city"]      # city is inside
        "address"
        print("----- User Details -----")  # 5. Display clearly
        print("Name   :", name)
        print("Email  :", email)
        print("City   :", city)
        print("-----")
    else:
        print("Failed to fetch user.")

show_user(1)
show_user(2)    # repeat with users/2
```

Output:

```
Status Code: 200
----- User Details -----
Name   : Leanne Graham
Email  : Sincere@april.biz
City   : Gwenborough
-----
Status Code: 200
----- User Details -----
Name   : Ervin Howell
Email  : Shanna@melissa.tv
City   : Wisokyburgh
-----
```

Q4. Send Data Using a POST Request

```
import requests

url = "https://jsonplaceholder.typicode.com/users"
payload = {"name": "Sagar Kumar", "role": "Student"}
```

```
response = requests.post(url, json=payload)

print("Status Code:", response.status_code)
print("Response:", response.json())
```

Output:

```
Status Code: 201
Response: {'name': 'Sagar Kumar', 'role': 'Student', 'id': 11}
```

Inspection: Status **201 = Created**. The server returned our data plus a new **id**, which means it “created” the record. (JSONPlaceholder is a fake test API, so the data is not really saved.)

Why POST instead of GET for sending new data?

1. **Purpose:** GET is only for **reading** data. POST is made for **creating/sending** new data.
 2. **Data location:** GET sends data in the **URL**, which is visible, saved in browser history and limited in length. POST sends data in the **body**, which can be large and structured JSON.
 3. **Security:** Sensitive data (passwords, personal details) should not appear in the URL. The POST body is safer, especially over HTTPS.
 4. **Server rules:** GET should not change anything on the server. POST is expected to create something new.
-

Q5. Extract Data from a Nested JSON Response

```
response_data = {
    "id": "chat-123",
    "choices": [
        {
            "index": 0,
            "message": {
                "role": "assistant",
                "content": "Python is a simple and powerful
programming language."
            }
        }
    ]
}
```

```
choices_list = response_data["choices"]      # Step 1: dictionary -> list
first_item = choices_list[0]                 # Step 2: list -> first item (dict)
message = first_item["message"]              # Step 3: nested dictionary
content = message["content"]                 # Step 4: final text
```

```
# Same in one line:
# content = response_data["choices"][0]["message"]["content"]

print("=" * 40)
print("AI Response:")
print(content.strip())
print("=" * 40)
```

Output:

```
=====
AI Response:
Python is a simple and powerful programming language.
=====
```

How I moved through the structure:

1. **The dictionary:** response_data is a dictionary. I used the **key** "choices" to get its value.
2. **The list:** "choices" is a **list**. I used **index [0]** to take the first item.
3. **The nested dictionary:** The first item is a dictionary. I used the key "message" to get another dictionary inside it.
4. **The final content value:** Inside message, I used the key "content" to get the **final text string**, then printed it neatly with `.strip()` and separator lines.

Rule to remember: use ["key"] for dictionaries and [index] for lists.

— End of Module 5 Assignment —